4. 트랜잭션 생명주기 분석

4.1 PBH 트랜잭션 생성 과정 분석

4.1.1 개요

PBH(Priority Blockspace for Humans) 트랜잭션의 생성 과정은 일반적인 블록체인 트랜잭션과 달리 World ID 증명 생성과 PBH 페이로드 구성이라는 추가적인 단계를 포함합니다. 이 과정은 사용자의 신원을 검증하면서도 프라이버시를 보장하는 영지식 증명 기술을 활용합니다.

4.1.2 World ID 증명 생성 과정 상세 설명

신원 생성 및 커밋먼트

World ID 증명 생성의 첫 번째 단계는 사용자의 고유한 신원(Identity)을 생성하는 것입니다:

```
// 사용자의 비밀 키로부터 신원 생성
let identity = semaphore_rs::identity::Identity::from_secret(&mut secret_bytes

// 신원으로부터 커밋먼트 생성 (Merkle Tree에 포함될 값)
let commitment = identity.commitment();
```

이 과정에서 생성되는 주요 구성 요소들:

1. **Private Key**: 사용자만 알고 있는 비밀 키
2. **Identity Commitment**: 공개적으로 공유 가능한 커밋먼트 값
3. **Identity Nullifier**: 중복 사용 방지를 위한 nullifier 생성에 사용

Merkle Tree 구성 및 포함 증명

World ID 시스템은 검증된 모든 사용자의 커밋먼트를 Merkle Tree에 저장합니다:

```
// 30단계 깊이의 Poseidon Merkle Tree 생성
let mut tree = semaphore_rs::poseidon_tree::LazyPoseidonTree::new_with_dense_p
30, // TREE_DEPTH
0, // 초기 인덱스
&U256::ZERO, // 기본값
);

// 사용자 커밋먼트를 트리에 추가
tree = tree.update_with_mutation(user_index, &identity.commitment());

// 특정 사용자에 대한 Merkle Proof 생성
let merkle_proof = tree.proof(user_index);
```

Semaphore 증명 생성

실제 영지식 증명은 다음과 같은 공개 입력들과 함께 생성됩니다:

```
// 증명 생성에 필요한 공개 입력들
let external_nullifier_hash = EncodedExternalNullifier::from(external_nullifier
let signal = transaction_hash; // 트랜잭션 해시가 signal로 사용됨
let root = tree.root(); // Merkle Tree의 루트
```

```
// Semaphore 영지식 증명 생성
let proof = semaphore_rs::protocol::generate_proof(
&identity, // 사용자의 비밀 신원
&merkle_proof, // Merkle 포함 증명
external_nullifier_hash, // 인코딩된 external nullifier
signal, // 트랜잭션 해시
)?;


// Nullifier hash 생성 (중복 방지용)
let nullifier_hash = semaphore_rs::protocol::generate_nullifier_hash(
&identity,
external_nullifier_hash
);
```

이 증명은 다음을 보장합니다:

- 사용자가 검증된 World ID 집합에 포함되어 있음
- 실제 신원을 노출하지 않음
- 동일한 external nullifier로 중복 사용 불가능

4.1.3 Signal Hash 계산 방법과 용도 분석

Signal Hash의 역할

Signal hash는 Semaphore 프로토콜에서 특정 메시지나 행동에 대한 증명을 생성할 때 사용되는 핵심 요소입니다. World Chain에서는 **트랜잭션 해시**가 signal로 사용됩니다.

EIP-4337 UserOperation의 Signal Hash 계산

PBH 트랜잭션이 EIP-4337 UserOperation 형태로 제출되는 경우, signal hash는 다음과 같이 계산됩니다:

```
// UserOperation 해시 계산 (EIP-4337 표준)
pub fn hash_user_op(user_op: &UserOperation) -> U256 {
let packed = pack_user_op(user_op);
let hash = keccak256(packed);
U256::from_be_bytes(hash.0)
}

// UserOperation 패킹 과정
fn pack_user_op(user_op: &UserOperation) -> Vec<u8> {
// EIP-4337에 따른 UserOperation 필드들의 ABI 인코딩
abi_encode(&[
user_op.sender,
user_op.nonce,
keccak256(&user_op.init_code),
keccak256(&user_op.call_data),
user_op.call_gas_limit,
user_op.verification_gas_limit,
user_op.pre_verification_gas,
user_op.max_fee_per_gas,
user_op.max_priority_fee_per_gas,
```

```
keccak256(&user_op.paymaster_and_data),
])
}
```

Signal Hash의 검증 과정

생성된 signal hash는 증명 검증 시 다음과 같이 사용됩니다:

```
// 증명 검증 시 signal hash 사용
let  is_valid = verify_proof(
root, // Merkle root
nullifier_hash, // Nullifier hash
signal, // 계산된 signal hash
external_nullifier_hash, // External nullifier
&proof, // 생성된 증명
TREE_DEPTH, // Tree 깊이 (30)
)?;
```

Signal Hash의 보안적 의미

Signal hash는 다음과 같은 보안적 특성을 제공합니다:

1. **트랜잭션 바인딩**: 증명이 특정 트랜잭션에만 유효하도록 보장
2. **재사용 방지**: 동일한 증명을 다른 트랜잭션에 사용할 수 없음
3. **무결성 보장**: 트랜잭션 내용이 변경되면 signal hash도 변경되어 증명이 무효화됨

4.1.4 External Nullifier 인코딩 방식 설명

External Nullifier의 구조

External Nullifier는 PBH 트랜잭션의 시간적 제약과 사용량 제한을 구현하는 핵심 구성 요소입니다:

```
#[derive(Builder, Debug, Default, Clone, Copy, PartialEq, Eq)]
pub struct ExternalNullifier {
pub version: Prefix, // 버전 (현재 v1 = 1)
pub year: u16, // 연도 (예: 2024)
pub month: u8, // 월 (1-12)
pub nonce: u16, // 월별 사용 횟수 (0부터 시작)
}
```

256비트 인코딩 방식

External Nullifier는 다음과 같은 비트 레이아웃으로 인코딩됩니다:

```
External Nullifier 인코딩 (256비트):
┌──────────────────────────────────────────────────────────────┐
| Bits 255-48: Empty (208 bits) - 향후 확장을 위한 예약 공간 |
├──────────────────────────────────────────────────────────────┤
| Bits 47-32: Year (16 bits) |
| 예: 2024 → 0x07E8 |
├──────────────────────────────────────────────────────────────┤
| Bits 31-24: Month (8 bits) |
| 예: 12월 → 0x0C |
├──────────────────────────────────────────────────────────────┤
```

```
| Bits 23-8: Nonce (16 bits) |
| 예: 5번째 사용 → 0x0005 |
├──────────────────────────────────────────────────────────────┤
| Bits 7-0: Version (8 bits) |
| 현재 버전 → 0x01 |
└──────────────────────────────────────────────────────────────┘
```

인코딩 구현

```rust
impl From<ExternalNullifier> for EncodedExternalNullifier {
fn  from(e: ExternalNullifier) -> Self {
EncodedExternalNullifier(U256::from(
(e.year as u64) << 32  // 32비트 왼쪽 시프트
| (e.month as u64) << 24  // 24비트 왼쪽 시프트
| (e.nonce as u64) << 8  // 8비트 왼쪽 시프트
| e.version as u64, // 최하위 8비트
))
}
}
```

디코딩 구현

```rust
impl TryFrom<EncodedExternalNullifier> for ExternalNullifier {
type Error = alloy_rlp::Error;
```

```
fn try_from(value: EncodedExternalNullifier) -> Result<Self, Self::Error> {
// 상위 208비트가 0인지 확인 (유효성 검사)
if value.0 > U256::from(1) << 48 {
return Err(alloy_rlp::Error::Custom("invalid external nullifier"));
}

let word: u64 = value.0.to();
// 비트 마스킹을 통한 각 필드 추출
let year = (word >> 32) as u16; // 상위 16비트
let month = ((word >> 24) & 0xFF) as u8; // 다음 8비트
let nonce = ((word >> 8) & 0xFFFF) as u16; // 다음 16비트
let version = (word & 0xFF) as u8; // 하위 8비트

// 버전 검증
if version != Prefix::V1 as u8 {
return Err(alloy_rlp::Error::Custom(
"invalid external nullifier version",
));
}

Ok(Self {
version: Prefix::V1,
```

```
year,
month,
nonce,
})
}
}
```

Date Marker와의 연동

External Nullifier는 현재 날짜와의 비교를 위해 `DateMarker` 와 연동됩니다:

```
impl ExternalNullifier {
// 현재 날짜로부터 External Nullifier 생성
pub fn with_date_marker(marker: DateMarker, nonce: u16) -> Self {
Self::v1(marker.month as u8, marker.year as u16, nonce)
}

// External Nullifier로부터 DateMarker 추출
pub fn date_marker(&self) -> DateMarker {
DateMarker::new(self.year as i32, self.month as u32)
}
}

// 현재 시간으로부터 DateMarker 생성
```

```rust
impl From<chrono::DateTime<chrono::Utc>> for DateMarker {
fn from(date: chrono::DateTime<chrono::Utc>) -> Self {
DateMarker::new(date.year(), date.month())
}
}
```

실제 사용 예시

```rust
// 2024년 12월, 5번째 PBH 트랜잭션 생성
let now = chrono::Utc::now();
let date_marker = DateMarker::from(now);
let external_nullifier = ExternalNullifier::with_date_marker(date_marker, 5);

// 인코딩 결과
let encoded = EncodedExternalNullifier::from(external_nullifier);
println!("Encoded: 0x{:064x}", encoded.0);
// 출력: 0x0000000000000000000000000000000000000000000000007e80c0501

// 검증 과정에서 사용
let signal = transaction_hash;
let proof = generate_proof(&identity, &merkle_proof, encoded.0, signal)?;
```

External Nullifier의 보안적 의미

1. **월별 제한**: 동일한 연도/월 조합으로 제한된 횟수만 사용 가능

2. **시간 제약**: 현재 월이 아닌 External Nullifier는 거부됨

3. **중복 방지**: 동일한 External Nullifier로 여러 증명 생성 불가

4. **확장성**: 208비트의 예약 공간으로 향후 기능 확장 가능

이러한 External Nullifier 인코딩 방식을 통해 World Chain은 사용자별 월간 PBH 트랜잭션 사용량을 효과적으로 제한하면서도, 시스템의 확장성과 보안성을 보장합니다.

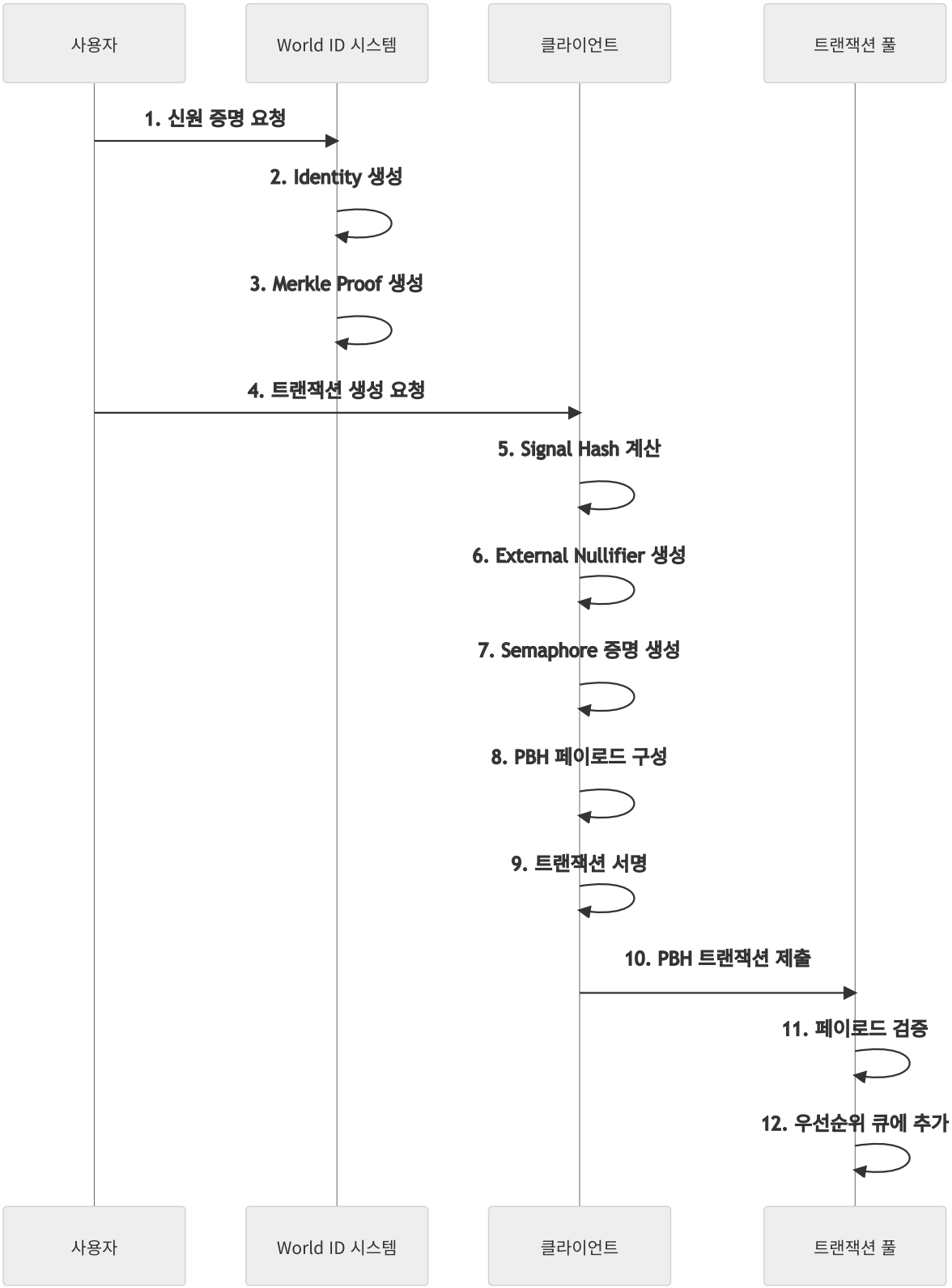
4.1.5 PBH 페이로드 구성

생성된 모든 구성 요소들은 최종적으로 PBH 페이로드로 조합됩니다:

```
let  pbh_payload = PBHPayload {

external_nullifier, // 월별 사용량 제한을 위한 식별자

nullifier_hash, // 중복 방지를 위한 해시

root, // Merkle Tree 루트

proof: Proof(proof), // Semaphore 영지식 증명

};
```

이 페이로드는 RLP 인코딩되어 트랜잭션의 일부로 포함되며, World Chain Builder에 의해 검증됩니다.

4.1.6 트랜잭션 생성 과정 요약


사용자
World ID 시스템
클라이언트
트랜잭션 풀
1. 신원 증명 요청
2. Identity 생성
3. Merkle Proof 생성
4. 트랜잭션 생성 요청
5. Signal Hash 계산
6. External Nullifier 생성
7. Semaphore 증명 생성
8. PBH 페이로드 구성
9. 트랜잭션 서명
10. PBH 트랜잭션 제출
11. 페이로드 검증
12. 우선순위 큐에 추가
사용자
World ID 시스템
클라이언트
트랜잭션 풀

이러한 복잡한 생성 과정을 통해 PBH 트랜잭션은 사용자의 실제 신원을 보호하면서도 검증된 인간 사용자임을 증명할 수 있으며, 월별 사용량 제한과 중복 방지 기능을 제공합니다.

4.2

트랜잭션 검증 과정 분석

4.2.1 개요

PBH 트랜잭션의 검증 과정은 World Chain Builder에서 수행되는 다단계 검증 시스템입니다. 이 과정은 일반적인 Ethereum 트랜잭션 검증에 추가하여 World ID 증명 검증, PBH 조건 확인, 그리고 Semaphore 프로토콜 검증을 포함합니다.

4.2.2 World Chain Builder의 검증 단계별 분석

1단계: 기본 트랜잭션 검증

모든 PBH 트랜잭션은 먼저 표준 Optimism 트랜잭션 검증을 통과해야 합니다:

```
// 기본 OP 트랜잭션 검증
let mut tx_outcome = self.inner.validate_one(origin, tx.clone()).await;
if !tx_outcome.is_valid() {
return tx_outcome; // 기본 검증 실패 시 즉시 반환
}
```

이 단계에서 검증되는 항목들:

- 트랜잭션 서명 유효성
- Nonce 순서 확인
- 가스 한도 및 가스 가격 검증

- 계정 잔액 확인
- EIP-1559 수수료 구조 검증

2단계: PBH 엔트리포인트 확인

트랜잭션의 대상 주소가 PBH 엔트리포인트인지 확인합니다:

```
// PBH 엔트리포인트 주소 확인
if transaction.to().unwrap_or_default() != self.pbh_entrypoint {
// 일반 트랜잭션으로 처리
return self.inner.validate_one(origin, transaction.clone()).await;
}

// PBH 트랜잭션으로 검증 진행
self.validate_pbh(origin, transaction).await
```

3단계: 가스 한도 검증

PBH 트랜잭션의 가스 한도가 설정된 최대값을 초과하지 않는지 확인합니다:

```
// PBH 트랜잭션 가스 한도 검증
if tx.gas_limit() > self.max_pbh_gas_limit.load(Ordering::Relaxed) {
return WorldChainPoolTransactionError::from(
PBHValidationError::PbhGasLimitExceeded
).to_outcome(tx);
}
```

4단계: 함수 시그니처 확인

트랜잭션의 calldata에서 함수 시그니처를 추출하여 PBH 관련 함수인지 확인합니다:

```
// 함수 시그니처 추출
let function_signature: [u8; 4] = tx
.input()
.get(..4)
.and_then(|bytes| bytes.try_into().ok())
.unwrap_or_default();


// PBH 번들 트랜잭션 확인
match function_signature {
IPBHEntryPoint::handleAggregatedOpsCall::SELECTOR => {
self.validate_pbh_bundle(origin, tx).await
}
_ => self.inner.validate_one(origin, tx.clone()).await,
}
```

4.2.3 PBH 조건 검사 과정 설명

Calldata 디코딩 및 구조 검증

PBH 번들 트랜잭션의 calldata를 디코딩하여 구조적 유효성을 확인합니다:

```
// handleAggregatedOps 호출 디코딩
```

```
let Ok(calldata) = IPBHEntryPoint::handleAggregatedOpsCall::abi_decode(tx.input(
return WorldChainPoolTransactionError::from(
PBHValidationError::InvalidCalldata
).to_outcome(tx);
};
```

시그니처 집계자 검증

모든 UserOperation이 올바른 PBH 시그니처 집계자를 사용하는지 확인합니다:

```
// 모든 집계된 연산에서 시그니처 집계자 확인
if !calldata._0.iter().all(|aggregator| {
aggregator.aggregator == self.pbh_signature_aggregator
}) {
return WorldChainPoolTransactionError::from(
PBHValidationError::InvalidSignatureAggregator,
).to_outcome(tx);
}
```

PBH 페이로드 추출 및 검증

각 UserOperation에 대응하는 PBH 페이로드를 추출하고 검증합니다:

```
for aggregated_ops in calldata._0 {
// 시그니처에서 PBH 페이로드 디코딩
let buff = aggregated_ops.signature.as_ref();
```

```
let pbh_payloads = match <Vec<PBHPayload>>::abi_decode(buff) {
Ok(pbh_payloads) => pbh_payloads,
Err(_) => {
return WorldChainPoolTransactionError::from(
PBHValidationError::InvalidCalldata,
).to_outcome(tx)
}
};

// UserOperation과 PBH 페이로드 수량 일치 확인
if pbh_payloads.len() != aggregated_ops.userOps.len() {
return WorldChainPoolTransactionError::from(
PBHValidationError::MissingPbhPayload
).to_outcome(tx);
}
}
```

병렬 페이로드 검증

각 PBH 페이로드를 병렬로 검증하여 성능을 최적화합니다:

```
let payloads: Vec<PbhPayload> = match pbh_payloads
.into_par_iter() // 병렬 반복자 사용
.zip(aggregated_ops.userOps)
```

```
.map(|(payload, op)| {
// UserOperation 해시를 signal로 사용
let  signal = crate::eip4337::hash_user_op(&op);
// PBH 페이로드 변환
let Ok(payload) = PbhPayload::try_from(payload) else {
return Err(PBHValidationError::InvalidCalldata.into());
};
// 페이로드 검증 수행
payload.validate(
signal,
&valid_roots,
self.max_pbh_nonce.load(Ordering::Relaxed),
)?;
Ok::<PbhPayload, WorldChainPoolTransactionError>(payload)
})
.collect::<Result<Vec<PbhPayload>, WorldChainPoolTransactionError>>()
{
Ok(payloads) => payloads,
Err(err) => return  err.to_outcome(tx),
};
```

중복 Nullifier Hash 검사

동일한 번들 내에서 중복된 nullifier hash가 있는지 확인합니다:

```
let mut seen_nullifier_hashes = HashSet::new();


// 모든 페이로드에서 nullifier hash 중복 검사
for payload in &payloads {
if !seen_nullifier_hashes.insert(payload.nullifier_hash) {
return WorldChainPoolTransactionError::from(
PBHValidationError::DuplicateNullifierHash,
).to_outcome(tx);
}
}
```

4.2.4 Semaphore 증명 검증 방식 분석

유효한 Root 검증

먼저 PBH 페이로드의 Merkle root가 유효한 World ID root 목록에 포함되어 있는지 확인합니다:

```
pub fn validate_root(&self, valid_roots: &[Field]) -> Result<(), PBHValidation
if !valid_roots.contains(&self.root) {
return Err(PBHValidationError::InvalidRoot);
}
Ok(())
}
```

유효한 root 목록은 World Chain Root Validator에 의해 관리됩니다:

```
// Root Validator에서 유효한 root 목록 가져오기
let valid_roots = self.root_validator.roots();
```

External Nullifier 시간 검증

External nullifier가 현재 월과 일치하는지, 그리고 nonce가 허용 범위 내에 있는지 확인합니다:

```
pub fn validate_external_nullifier(
&self,
date: chrono::DateTime<chrono::Utc>,
pbh_nonce_limit: u16,
) -> Result<(), PBHValidationError> {
// 현재 월과 일치하는지 확인
if self.external_nullifier.date_marker() != DateMarker::from(date) {
return Err(PBHValidationError::InvalidExternalNullifierPeriod);
}

// Nonce 한도 확인
if self.external_nullifier.nonce >= pbh_nonce_limit {
return Err(PBHValidationError::InvalidExternalNullifierNonce);
}

Ok(())
```

```
}
```

압축된 증명 처리

World Chain은 효율성을 위해 압축된 Semaphore 증명을 지원합니다:

```
// 증명이 압축되었는지 확인
let flat = self.proof.0.flatten();
let proof = if (flat[4] | flat[5] | flat[6] | flat[7]).is_zero() {
// 압축된 증명 - 압축 해제 필요
let compressed_flat = [flat[0], flat[1], flat[2], flat[3]];
let compressed_proof =
semaphore_rs_proof::compression::CompressedProof::from_flat(compressed_flat);
&semaphore_rs_proof::compression::decompress_proof(compressed_proof)
.ok_or(PBHValidationError::InvalidProof)?
} else {
// 일반 증명
&self.proof.0
};
```

핵심 Semaphore 증명 검증

최종적으로 Semaphore 프로토콜의 `verify_proof` 함수를 사용하여 영지식 증명을 검증합니다:

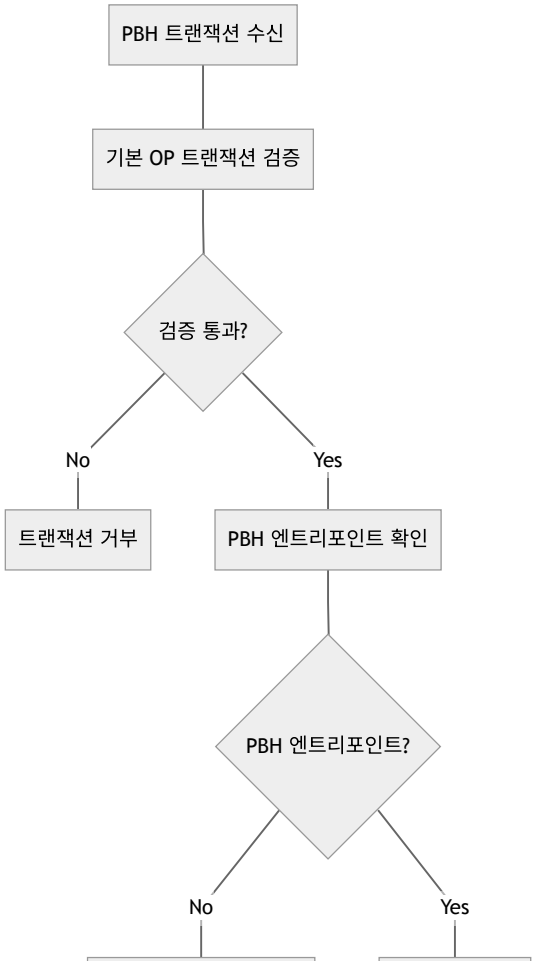
```
// Semaphore 증명 검증
if verify_proof(
```

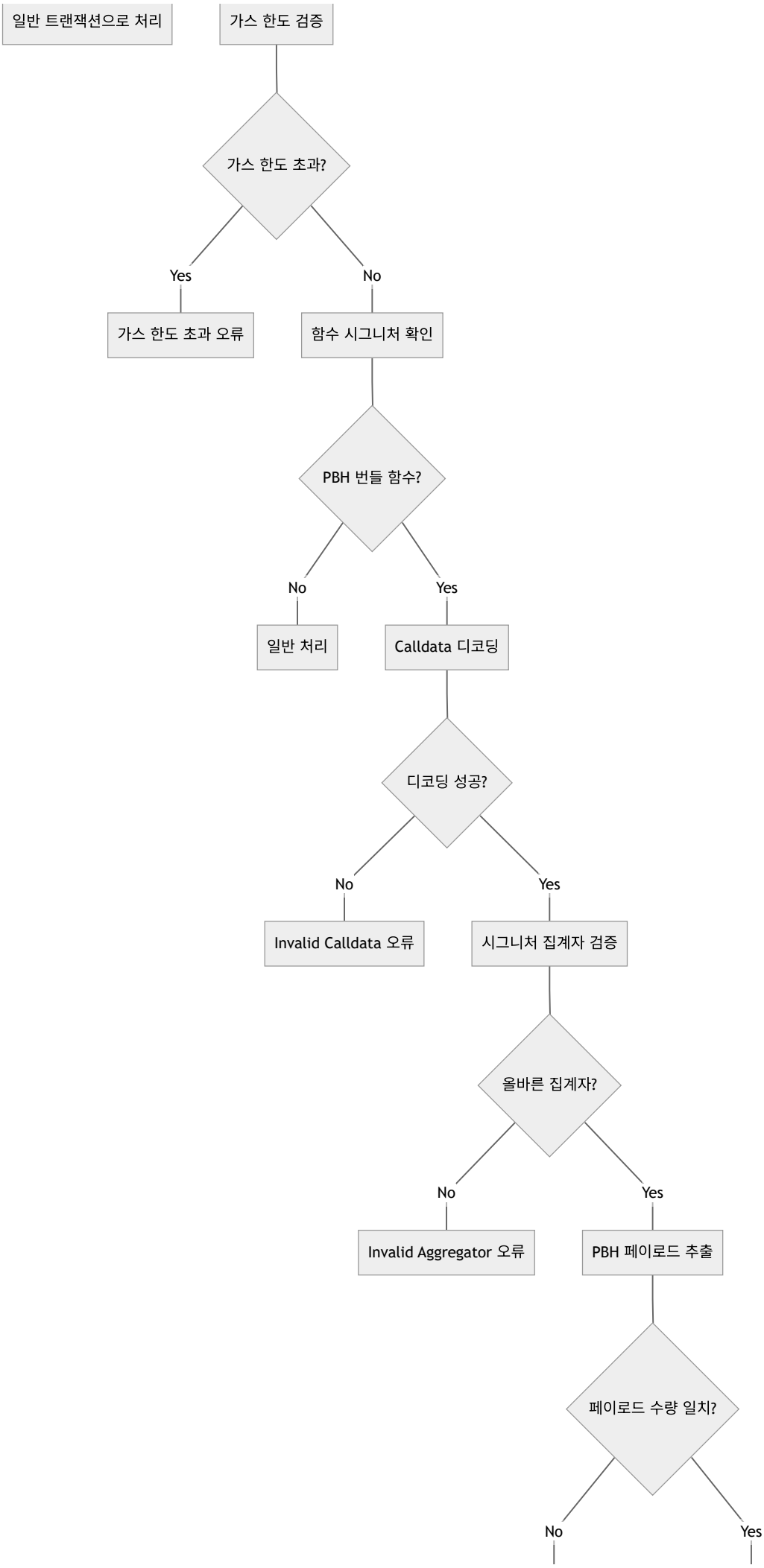
```
self.root, // Merkle root
self.nullifier_hash, // Nullifier hash
signal, // UserOperation 해시
EncodedExternalNullifier::from(self.external_nullifier).0, // External nullifier
proof, // 증명
TREE_DEPTH, // Tree 깊이 (30)
)? {
Ok(())
} else {
Err(PBHValidationError::InvalidProof)
}
```

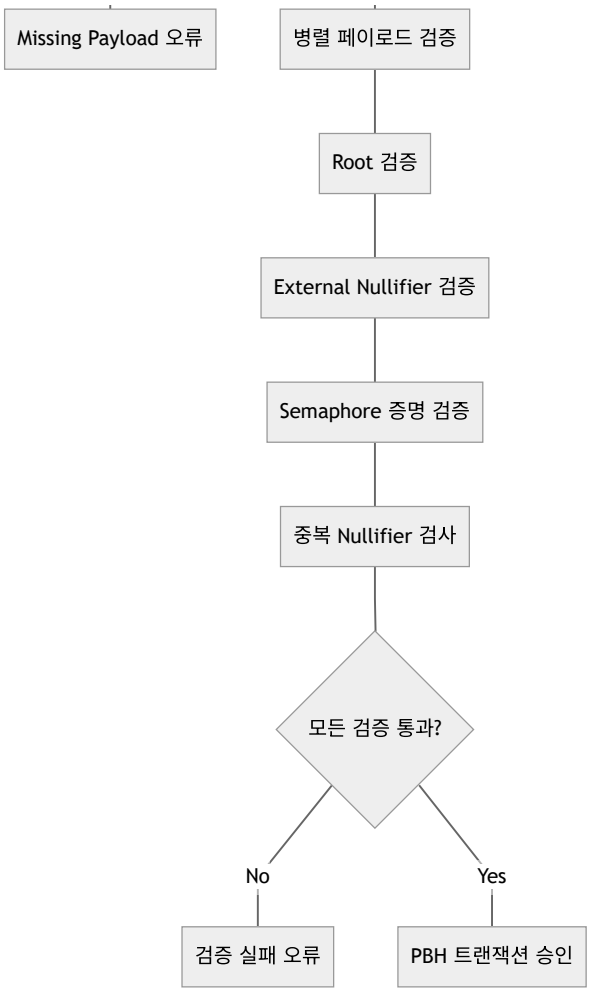
이 검증 과정에서 확인되는 사항들:

1. **포함 증명**: 사용자가 유효한 World ID 집합에 포함되어 있음
2. **Signal 바인딩**: 증명이 특정 UserOperation에 바인딩되어 있음
3. **Nullifier 유효성**: 생성된 nullifier hash가 올바름
4. **External Nullifier 일치**: 증명에 사용된 external nullifier가 일치함
5. **영지식 특성**: 실제 신원 정보를 노출하지 않음

4.2.5 검증 과정 흐름도


PBH 트랜잭션 수신
기본 OP 트랜잭션 검증
검증 통과?
No
Yes
트랜잭션 거부
PBH 엔트리포인트 확인
PBH 엔트리포인트?
No
Yes


일반 트랜잭션으로 처리
가스 한도 검증
가스 한도 초과?
Yes
No
가스 한도 초과 오류
함수 시그니처 확인
PBH 번들 함수?
No
Yes
일반 처리
Calldata 디코딩
디코딩 성공?
No
Yes
Invalid Calldata 오류
시그니처 집계자 검증
올바른 집계자?
No
Yes
Invalid Aggregator 오류
PBH 페이로드 추출
페이로드 수량 일치?
No
Yes


Missing Payload 오류
병렬 페이로드 검증
Root 검증
External Nullifier 검증
Semaphore 증명 검증
중복 Nullifier 검사
모든 검증 통과?
No
Yes
검증 실패 오류
PBH 트랜잭션 승인

4.2.6 검증 실패 시 오류 처리

각 검증 단계에서 발생할 수 있는 오류들과 그 처리 방식:

```
#[derive(Error, Debug)]
pub  enum PBHValidationError {
#[error("Invalid root")]
InvalidRoot,
#[error("Invalid external nullifier period")]
InvalidExternalNullifierPeriod,
#[error("Invalid external nullifier nonce")]
InvalidExternalNullifierNonce,
#[error("Invalid proof")]
InvalidProof,
#[error("Invalid calldata encoding")]
InvalidCalldata,
#[error("Missing PBH Payload")]
MissingPbhPayload,
#[error("InvalidSignatureAggregator")]
InvalidSignatureAggregator,
#[error("PBH gas limit exceeded")]
PbhGasLimitExceeded,
#[error("Duplicate nullifier hash")]
DuplicateNullifierHash,
```

```
}
```

4.2.7 성능 최적화

검증 과정의 성능을 최적화하기 위한 방법들:

1. **병렬 처리**: 여러 PBH 페이로드를 동시에 검증
2. **캐싱**: 유효한 root 목록을 메모리에 캐시
3. **조기 종료**: 첫 번째 검증 실패 시 즉시 반환
4. **압축된 증명**: 네트워크 대역폭과 저장 공간 절약

이러한 다단계 검증 과정을 통해 World Chain은 PBH 트랜잭션의 무결성과 보안성을 보장하면서도, 효율적인 처리 성능을 제공합니다.##

4.3 블록 생성 및 포함 과정 분석

4.3.1 개요

World Chain의 블록 생성 과정은 기존 Optimism 블록 빌더를 확장하여 PBH(Priority Blockspace for Humans) 트랜잭션에 우선순위를 부여하는 시스템입니다. 이 과정은 검증된 블록스페이스 용량 관리, 우선순위 기반 트랜잭션 정렬, 그리고 nullifier hash 관리를 포함합니다.

4.3.2 우선순위 정책에 따른 트랜잭션 정렬 방식 설명

World Chain 트랜잭션 우선순위 구조

World Chain은 다음과 같은 우선순위 구조를 사용합니다:

```
#[derive(Debug, Default, Clone, Ord, PartialOrd, Eq, PartialEq)]

pub  struct WorldChainPriority {

is_pbh: bool, // PBH 트랜잭션 여부 (최우선)

effective_tip_per_gas: Option<U256>, // 가스당 팁 (2차 우선순위)
```

```
}
```

이 구조에서 중요한 점은 **필드 순서**입니다. Rust의 자동 파생된 `Ord` 구현은 필드 순서대로 비교하므로:

1. **1차 우선순위**: `is_pbh` - PBH 트랜잭션이 항상 일반 트랜잭션보다 우선
2. **2차 우선순위**: `effective_tip_per_gas` - 동일한 타입 내에서는 가스 팁으로 정렬

트랜잭션 우선순위 계산

```
impl<T> TransactionOrdering for WorldChainOrdering<T>
where
T: WorldChainPoolTransaction + 'static,
{
type PriorityValue = WorldChainPriority;
type Transaction = T;

fn priority(
&self,
transaction: &Self::Transaction,
base_fee: u64,
) -> Priority<Self::PriorityValue> {
let effective_tip_per_gas = transaction
.effective_tip_per_gas(base_fee)
.map(U256::from);
```

```
Some(WorldChainPriority {
is_pbh: transaction.pbh_payload().is_some(), // PBH 페이로드 존재 여부
effective_tip_per_gas,
})
.into()
}
}
```

우선순위 비교 예시

```
// PBH 트랜잭션은 항상 일반 트랜잭션보다 우선
let  pbh_tx = WorldChainPriority {
is_pbh: true,
effective_tip_per_gas: Some(U256::from(100)), // 낮은 팁
};

let  regular_tx = WorldChainPriority {
is_pbh: false,
effective_tip_per_gas: Some(U256::from(10000)), // 높은 팁
};

assert!(pbh_tx > regular_tx); // PBH가 우선
```

```
// 동일한 타입 내에서는 가스 팁으로 정렬
let high_tip_pbh = WorldChainPriority {
is_pbh: true,
effective_tip_per_gas: Some(U256::from(1000)),
};

let low_tip_pbh = WorldChainPriority {
is_pbh: true,
effective_tip_per_gas: Some(U256::from(100)),
};

assert!(high_tip_pbh > low_tip_pbh); // 높은 팁이 우선
```

4.3.3 PBH 블록스페이스 할당 및 관리 방식 분석

검증된 블록스페이스 용량 설정

World Chain Builder는 `verified_blockspace_capacity` 설정을 통해 PBH 전용 블록스페이스를 관리합니다:

```
// PBH 전용 가스 한도 계산
let verified_gas_limit = (self.verified_blockspace_capacity as u64 * gas_limit)

// 예시: 블록 가스 한도 30M, PBH 용량 40%
```

```
// verified_gas_limit = (40 * 30,000,000) / 100 = 12,000,000 가스
```

트랜잭션 실행 중 블록스페이스 관리

블록 생성 과정에서 PBH 트랜잭션은 별도의 가스 한도 내에서 관리됩니다:

```
// PBH 트랜잭션 가스 한도 확인
if  let Some(payloads) = pooled_tx.pbh_payload() {
// 현재 사용량 + 새 트랜잭션이 PBH 한도를 초과하는지 확인
if  info.cumulative_gas_used + tx.gas_limit() > verified_gas_limit {
best_txs.mark_invalid(tx.signer(), tx.nonce());
continue; // PBH 블록스페이스 초과 시 건너뛰기
}
// Nullifier hash 중복 검사
if  payloads.iter().any(|payload|
!spent_nullifier_hashes.insert(payload.nullifier_hash)
) {
best_txs.mark_invalid(tx.signer(), tx.nonce());
invalid_txs.push(*pooled_tx.hash());
continue;
}
}
```

동적 가스 한도 조정

PBH 트랜잭션이 실행될 때마다 남은 가스 한도가 동적으로 조정됩니다:

```
let gas_used = match builder.execute_transaction(tx.clone()) {
Ok(res) => {
if let Some(payloads) = pooled_tx.pbh_payload() {
// 첫 번째 PBH 트랜잭션인 경우 고정 가스 차감
if spent_nullifier_hashes.len() == payloads.len() {
gas_limit -= FIXED_GAS; // 100,000 가스
}
// 각 nullifier hash당 COLD_SSTORE_GAS 차감
gas_limit -= COLD_SSTORE_GAS * payloads.len() as u64; // 20,000 * 개수
}
res
}
Err(err) => {
// 오류 처리...
}
};
```

블록스페이스 할당 시각화

```
블록 가스 한도: 30,000,000 (100%)
┌──────────────────────────────────────────────────────────────┐
| PBH 블록스페이스 (40% = 12,000,000 가스) |
├──────────────────────────────────────────────────────────────┤
```

```
| 🟢 PBH Tx 1: 2,000,000 가스 |
| 🟢 PBH Tx 2: 1,500,000 가스 |
| 🟢 PBH Tx 3: 3,000,000 가스 |
| 🟢 PBH Tx 4: 2,500,000 가스 |
| 🟢 PBH Tx 5: 1,800,000 가스 |
| 🟢 PBH Tx 6: 1,200,000 가스 |
| 사용량: 12,000,000 / 12,000,000 (100%) |
├──────────────────────────────────────────────────────────────┤
| 일반 블록스페이스 (60% = 18,000,000 가스) |
├──────────────────────────────────────────────────────────────┤
| 일반 Tx 1: 500,000 가스 |
| 일반 Tx 2: 800,000 가스 |
| 일반 Tx 3: 1,200,000 가스 |
| ... (가스비 순으로 정렬) |
| 사용량: 18,000,000 / 18,000,000 (100%) |
└──────────────────────────────────────────────────────────────┘
```

4.3.4 Nullifier Hash 관리 시스템

Spent Nullifier Hash 추적

블록 생성 과정에서 사용된 nullifier hash들을 추적하여 중복 사용을 방지합니다:

```
let mut spent_nullifier_hashes = HashSet::new();
```

```
// 각 PBH 트랜잭션 처리 시
if  let Some(payloads) = pooled_tx.pbh_payload() {
// 중복 nullifier hash 검사
if  payloads.iter().any(|payload|
!spent_nullifier_hashes.insert(payload.nullifier_hash)
) {
// 중복 발견 시 트랜잭션 무효화
best_txs.mark_invalid(tx.signer(), tx.nonce());
invalid_txs.push(*pooled_tx.hash());
continue;
}
}
```

Nullifier Hash 소비 트랜잭션 생성

블록 끝에서 사용된 모든 nullifier hash를 PBH 엔트리포인트에 기록하는 트랜잭션을 생성합니다:

```
// 사용된 nullifier hash가 있는 경우에만 실행
if !spent_nullifier_hashes.is_empty() {
let  tx = spend_nullifiers_tx(self, builder.evm_mut(), spent_nullifier_hashes)
.map_err(|e| {
error!(target: "payload_builder", %e, "failed to build spend nullifiers transact
PayloadBuilderError::Other(e.into())
})?;
```

```
// Nullifier 소비 트랜잭션 실행 시도
match builder.execute_transaction(tx.clone()) {
Ok(gas_used) => self.commit_changes(info, base_fee, gas_used, tx),
Err(e) => {
error!(target: "payload_builder", %e, "spend nullifiers transaction failed")
// 실패해도 PBH 트랜잭션은 포함됨 (우선순위 보장)
}
}
}
```

Nullifier 소비 트랜잭션 구성

```
pub fn spend_nullifiers_tx<DB, EVM, Client, Pool>(
ctx: &WorldChainPayloadBuilderCtx<Client, Pool>,
evm: &mut EVM,
nullifier_hashes: HashSet<Field>,
) -> eyre::Result<Recovered<OpTransactionSigned>>
where
// ... 타입 제약 조건들
{
// Builder 계정의 현재 nonce 조회
let nonce = evm
.db_mut()
```

```
.basic(ctx.builder_private_key.address())?
.unwrap_or_default()
.nonce;

// 트랜잭션 구성
let mut tx = OpTransactionRequest::default()
.nonce(nonce)
.gas_limit(dyn_gas_limit(nullifier_hashes.len() as u64)) // 동적 가스 한도
.max_priority_fee_per_gas(evm.block().basefee.into())
.max_fee_per_gas(evm.block().basefee.into())
.with_chain_id(evm.chain_id())
.with_call(&spendNullifierHashesCall {
_nullifierHashes: nullifier_hashes.into_iter().collect(),
})
.to(ctx.pbh_entry_point)
.build_typed_tx()
.map_err(|e| eyre!("{:?}", e))?;

// Builder 개인키로 트랜잭션 서명
let signature = ctx.builder_private_key.sign_transaction_sync(&mut tx)?;
let signed: OpTransactionSigned = tx.into_signed(signature).into();
Ok(signed.try_into_recovered_unchecked()?)
```

```
}
```

동적 가스 한도 계산

Nullifier 소비 트랜잭션의 가스 한도는 nullifier hash 개수에 따라 동적으로 계산됩니다:

```
pub const COLD_SSTORE_GAS: u64 = 20000; // 각 nullifier hash당 가스
pub const FIXED_GAS: u64 = 100_000; // 기본 실행 가스

pub const fn dyn_gas_limit(len: u64) -> u64 {
FIXED_GAS + len * COLD_SSTORE_GAS
}

// 예시: 5개의 nullifier hash
// 가스 한도 = 100,000 + 5 * 20,000 = 200,000 가스
```

4.3.5 폴백 메커니즘과 기본 실행 클라이언트 연동 설명

폴백 메커니즘 개요

World Chain Builder는 다음과 같은 상황에서 폴백 메커니즘을 사용합니다:

1. **PBH 트랜잭션 부족**: PBH 블록스페이스가 완전히 채워지지 않은 경우
2. **Builder 실패**: Nullifier 소비 트랜잭션 실행 실패
3. **검증 실패**: PBH 트랜잭션 검증 실패 시

PBH 블록스페이스 미사용 시 폴백

PBH 트랜잭션이 할당된 블록스페이스를 모두 사용하지 못한 경우, 남은 공간을 일반 트랜잭션으로 채웁니다:

```
// PBH 블록스페이스 사용량 확인
let pbh_gas_used = info.cumulative_gas_used.min(verified_gas_limit);
let remaining_pbh_space = verified_gas_limit.saturating_sub(pbh_gas_used);

// 남은 PBH 공간을 일반 트랜잭션 한도에 추가
let total_available_gas = gas_limit; // 전체 블록 가스 한도
let regular_gas_limit = total_available_gas - pbh_gas_used;

// 일반 트랜잭션으로 남은 공간 채우기
while let Some(regular_tx) = regular_tx_iterator.next() {
if info.cumulative_gas_used + regular_tx.gas_limit() <= total_available_gas {
// 트랜잭션 실행
builder.execute_transaction(regular_tx)?;
}
}
```

Nullifier 소비 실패 시 폴백

Nullifier 소비 트랜잭션이 실패하더라도 PBH 트랜잭션들은 블록에 포함됩니다:

```
// Nullifier 소비 트랜잭션 실행 시도
match builder.execute_transaction(tx.clone()) {
```

```
Ok(gas_used) => {
// 성공 시 정상 처리
self.commit_changes(info, base_fee, gas_used, tx);
}
Err(e) => {
// 실패해도 PBH 트랜잭션은 이미 포함됨
error!(target: "payload_builder", %e, "spend nullifiers transaction failed");
// 폴백: PBH 트랜잭션은 우선순위를 유지하되,
// nullifier는 다음 블록에서 처리하거나 기본 실행 클라이언트에서 처리
}
}
```

기본 실행 클라이언트와의 연동

World Chain Builder가 실패하거나 사용할 수 없는 경우, 기본 Optimism 실행 클라이언트로 폴백됩니다:

```
// Builder 설정에서 폴백 동작 정의
impl<Client, S, T> PayloadBuilder for WorldChainPayloadBuilder<Client, S, T> {
fn  on_missing_payload(
&self,
_args: BuildArguments<Self::Attributes, Self::BuiltPayload>,
) -> MissingPayloadBehaviour<Self::BuiltPayload> {
// 진행 중인 작업을 기다림 (경쟁 방지)
MissingPayloadBehaviour::AwaitInProgress
```

```
}

}
```

폴백 시나리오별 처리

시나리오 1: PBH 트랜잭션 부족

```
PBH 할당: 12M 가스

실제 PBH: 8M 가스 사용

폴백 처리: 남은 4M 가스를 일반 트랜잭션으로 채움

결과: 총 30M 가스 모두 사용 (8M PBH + 22M 일반)
```

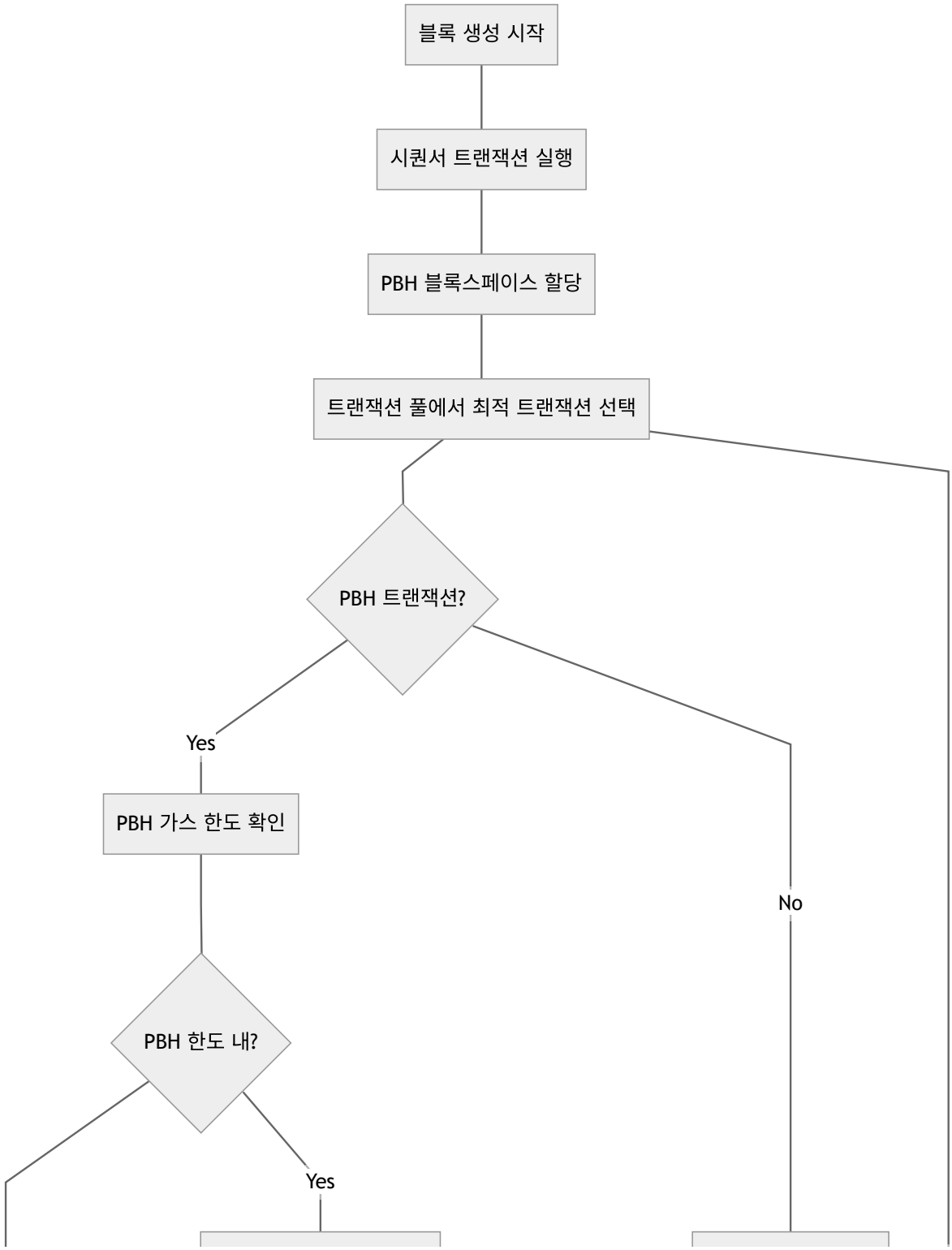
시나리오 2: Nullifier 소비 실패

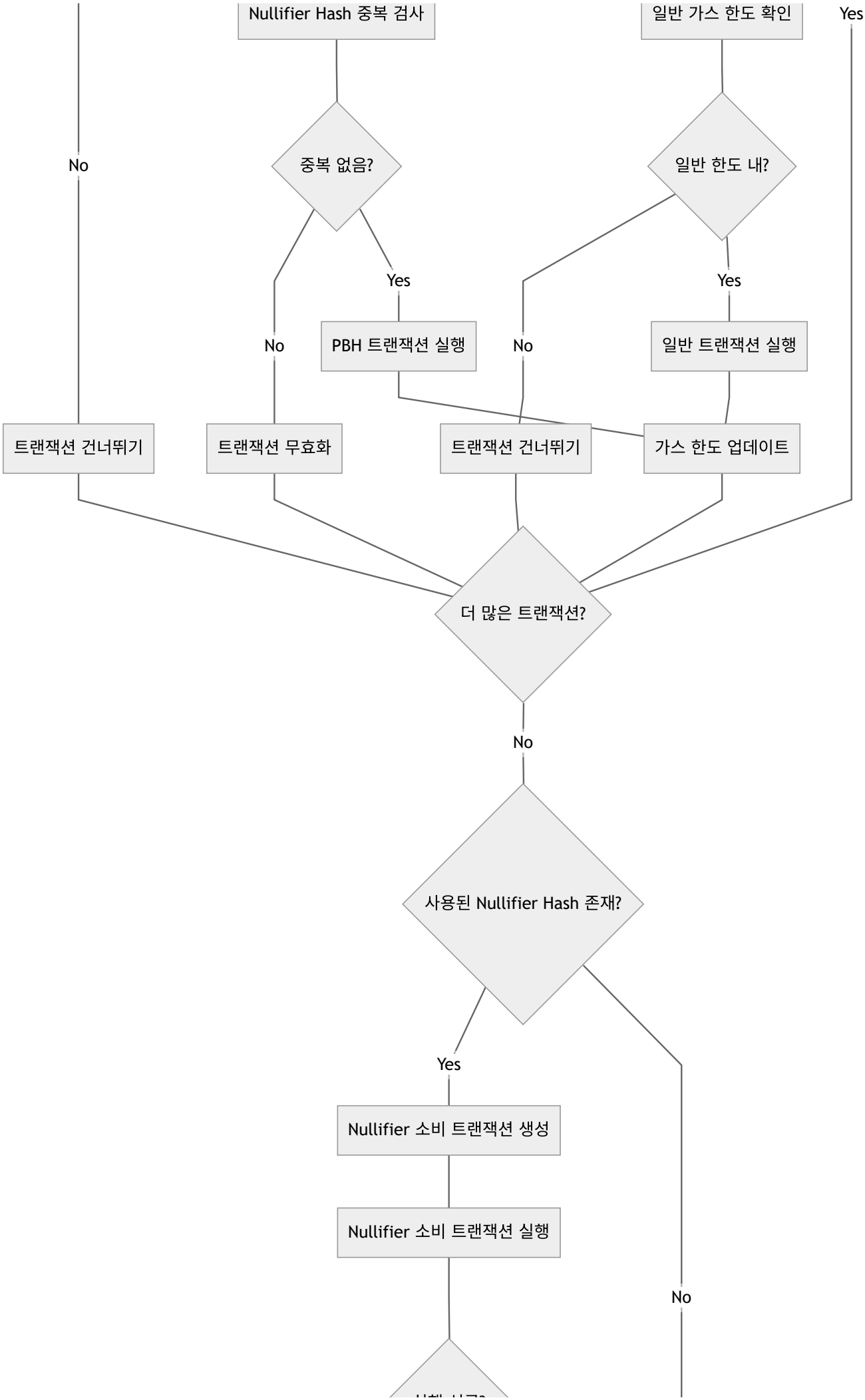
```
PBH 트랜잭션: 정상 실행 및 포함

Nullifier 소비: 실행 실패 (잔액 부족 등)

폴백 처리: PBH 트랜잭션은 블록에 포함, nullifier는 미처리 상태

결과: 다음 블록에서 nullifier 처리 재시도
```

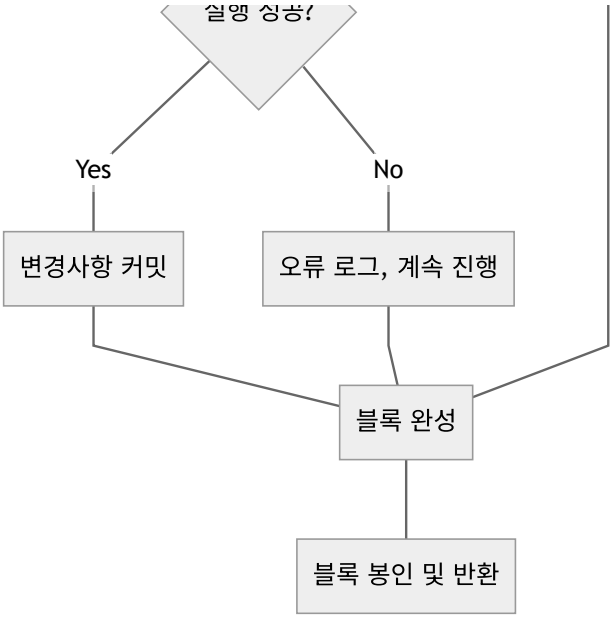
시나리오 3: Builder 완전 실패

```
World Chain Builder: 전체 실패

폴백 처리: 기본 Optimism 실행 클라이언트 사용

결과: PBH 우선순위 없이 일반 가스비 기반 정렬
```

4.3.6 블록 생성 과정 전체 흐름


블록 생성 시작
시퀀서 트랜잭션 실행
PBH 블록스페이스 할당
트랜잭션 풀에서 최적 트랜잭션 선택
PBH 트랜잭션?
Yes
PBH 가스 한도 확인
No
PBH 한도 내?
Yes


Nullifier Hash 중복 검사
일반 가스 한도 확인
Yes
No
중복 없음?
일반 한도 내?
Yes
Yes
No
PBH 트랜잭션 실행
No
일반 트랜잭션 실행
트랜잭션 건너뛰기
트랜잭션 무효화
트랜잭션 건너뛰기
가스 한도 업데이트
더 많은 트랜잭션?
No
사용된 Nullifier Hash 존재?
Yes
Nullifier 소비 트랜잭션 생성
Nullifier 소비 트랜잭션 실행
No



4.3.7 성능 최적화 및 모니터링

성능 최적화 기법

1. **병렬 트랜잭션 검증**: 여러 PBH 트랜잭션을 동시에 검증
2. **조기 종료**: 가스 한도 초과 시 즉시 중단
3. **캐시 활용**: 유효한 root 목록과 설정값 캐싱
4. **배치 처리**: Nullifier hash를 배치로 처리

모니터링 메트릭

```
// 블록 생성 과정에서 수집되는 메트릭들
debug!(target: "payload_builder",
```

```
id=%op_ctx.payload_id(),

parent_header = ?ctx.inner.parent().hash(),

parent_number = ctx.inner.parent().number,

"building new payload"

);



debug!(target: "payload_builder",

id=%op_ctx.payload_id(),

sealed_block_header = ?sealed_block.header(),

"sealed built block"

);
```

이러한 복잡한 블록 생성 과정을 통해 World Chain은 PBH 트랜잭션에 우선순위를 부여하면서도, 전체 네트워크의 효율성과 안정성을 보장합니다. 폴백 메커니즘을 통해 예외 상황에서도 블록 생성이 중단되지 않도록 하여, 네트워크의 연속성을 유지합니다.